For example, suppose 1µsec timer precision is required. The user could supply an external 2-25MHz clock in place of the LPOSC and program both the LPOSC and XOSC frequency registers in MHz units rather than kHz. The maximum frequency of the external clock is 29MHz.

12.10.9. List of registers

The AON Timer shares a register address space with the power management subsystems in the always-on domain. The address space is referred to as POWMAN elsewhere in this document and a complete list of POWMAN registers is provided in Section 6.4. The registers associated with the AON Timer are:

- SET_TIME_63TO48
- SET_TIME_47TO32
- SET_TIME_31TO16
- SET_TIME_15TO0
- READ_TIME_UPPER
- READ_TIME_LOWER
- ALARM_TIME_63TO48
- ALARM_TIME_47TO32
- ALARM_TIME_31TO16
- ALARM_TIME_15TO0
- TIMER

12.11. HSTX

The high-speed serial transmit (HSTX) streams data from the system clock domain to up to 8 GPIOs at a rate independent of the system clock. On RP2350, GPIOs 12 through 19 are HSTX-capable. HSTX is output-only.

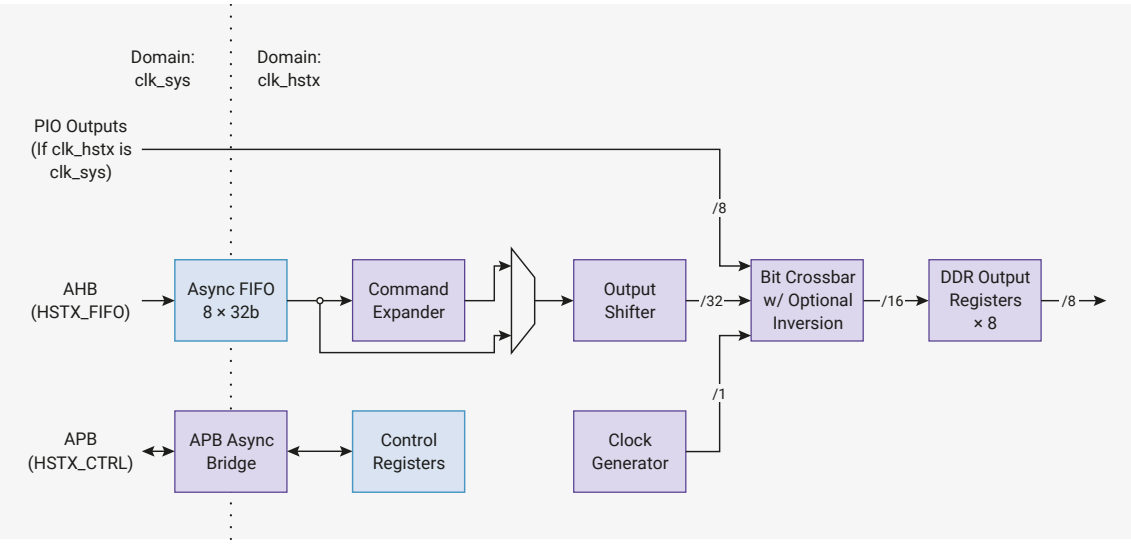

Figure 126. A 32-bit-wide asynchronous FIFO provides high-bandwidth access from the system DMA. The command expander manipulates the datastream, and the output shift register portions the 32-bit data over successive HSTX clock cycles, swizzled by the bit crossbar. Outputs are double-data-rate: two bits per pin per cycle.

HSTX drives data through GPIOs using DDR output registers to transfer up to two bits per clock cycle per pin. The HSTX balances all delays to GPIO outputs within 300 picoseconds, minimising common-mode components when using neighbouring GPIOs as a pseudo-differential driver. This also helps maintain destination setup and hold time when a clock is driven alongside the output data.

The maximum frequency for the HSTX clock is 150 MHz, the same as the system clock. With DDR output operation, this

is a maximum data rate of 300 Mb/s per pin. There are no limits on the frequency ratio of the system and HSTX clocks, however each clock must be individually fast enough to maintain your required throughput. Very low system clock frequencies coupled with very high HSTX frequencies might encounter system DMA bandwidth limitations, since the DMA is capped at one HSTX FIFO write per system clock cycle.

12.11.1. Data FIFO

An 8-entry, 32-bit-wide FIFO buffers data between the system clock domain (`clk_sys`) and the HSTX clock domain (`clk_hstx`). This is accessed through the AHB `FASTPERI` arbiter, providing single-cycle write access from the DMA. The FIFO status is also available through this same bus interface, for faster polled processor IO; see Section 12.11.8.

The FIFO is accessed through a bus interface separate from the control registers (Section 12.11.7), which take multiple cycles to access due to the asynchronous bus crossing. This design avoids incurring bus stalls on the system DMA or the `FASTPERI` arbiter when accessing the FIFO.

The HSTX side also pops 32 bits at a time from the FIFO. The word data stream from the FIFO is optionally manipulated by the command expander (Section 12.11.5) before being passed to the output shift register.

12.11.2. Output shift register

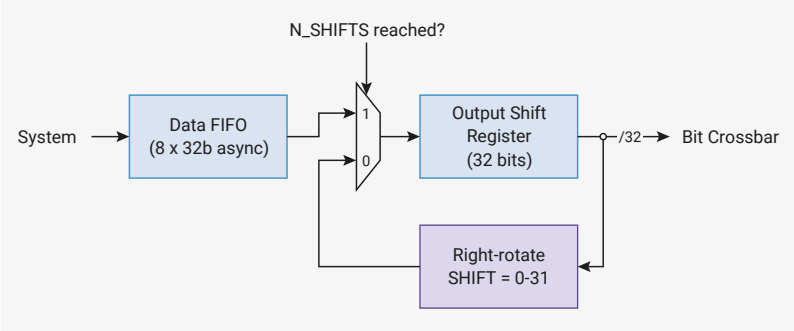

Figure 127. Every cycle, the output shift register either refills 32 bits from the FIFO or recirculates data through a right-rotate function. The rotate can be used to perform left or right shifts, and to repeat data.

The HSTX's internal data paths are 32 bits wide, but the output is narrower: no more than 16 bits can be output per HSTX cycle (8 GPIOs × DDR). The output shift register adapts these mismatched data widths. The output shift register is a 32-bit shift register, which always refills 32 bits at a time, either from the command expander output or directly from the data FIFO.

The source of data for the output shift register is configured by the CSR.EXPAND_EN field:

- when set, the command expander interposes the FIFO and the output shift register
- when clear, the command expander is bypassed, popping the FIFO directly into the shift register

Whenever CSR.EN is low, the shift register is flushed to empty. Once HSTX has been configured, and `EN` is set high, the shift register is ready to accept data, and will pop data as soon as it becomes available.

After popping the first data word, the shift register will now shift *every HSTX clock cycle* until it becomes empty. The shift behaviour is configured by:

- CSR.N_SHIFTS, which determines how many times to shift before the register is considered empty
- CSR.SHIFT, which is a *right-rotate* applied to the shift register every cycle

CSR.N_SHIFTS and CSR.SHIFT must only be changed when CSR.EN is low. It is safe to change these fields in the same register write that sets `EN` from low to high.

`SHIFT` × `N_SHIFTS` is not necessarily less than or equal to 32. For example, a `SHIFT` of 31 might be used to shift the register *left* by one bit per cycle, since right-rotate is a modular operation, and -1 is equal to 31 under a modulus of 32.

When the shift register is about to become empty, it will immediately refill with fresh data from the command expander or FIFO if data is available. When data is available, the shift register is never empty for any cycle. If data is not available,

the shift register becomes empty and stops shifting until more data is provided. Once data is provided, the shift register refills and begins shifting once again.

12.11.3. Bit crossbar

The bit crossbar controls which bits of the output shift register appear on which GPIOs during the first and second half of each HSTX clock cycle. There is a configuration register for each pin, BIT0 through BIT7:

- `BITx.SEL_P` selects which shift register bit (0 through 31) is output for the first half of each HSTX clock cycle
- `BITx.SEL_N` selects which shift register bit (0 through 31) is output for the second half of each clock cycle
- `BITx.INV` inverts the output (logical NOT)
- `BITx.CLK` indicates that this pin should be connected to the clock generator (Section 12.11.4) rather than the output shift register

To disable DDR behaviour set `SEL_N` equal to `SEL_P`. To implement a differential output, configure two pins identically except for the `INV` bit, which should be set for one pin and clear for the other.

12.11.3.1. Examples: one pin

Together with the `SHIFT` and `N_SHIFTS` controls for the shift register, the pin configuration determines the data layout passed through the HSTX. Since not all of us are accustomed to thinking in four dimensions, it's worth going through some examples with a single pin:

- `N_SHIFTS` = `32`, `SHIFT` = `1`, `SEL_P` = `0`, `SEL_N` = `0`:
  - Shift out one bit per HSTX clock cycle, LSB-first.
  - Each cycle, the shift register advances to the right by one, and the least-significant bit at that time is presented to the pin for both halves of the cycle, since `SEL_P` and `SEL_N` both select the same bit.
- `N_SHIFTS` = `32`, `SHIFT` = `31`, `SEL_P` = `31`, `SEL_N` = `31`:
  - Shift out one bit per HSTX clock cycle, MSB-first.
  - Each cycle, the shift register advances to the left by one (or rather, wraps around the right-hand edge of the register and ends up one bit left of where it started), and the most-significant bit at that time is presented to the pin.
- `N_SHIFTS` = `16`, `SHIFT` = `2`, `SEL_P` = `0`, `SEL_N` = `1`:
  - Shift out two bits per HSTX clock cycle, LSB-first.
  - Each cycle, the shift register advances to the right by two. The least-significant bit is presented to the pin for the first half of that cycle, and the neighbouring bit is presented for the second half.
- `N_SHIFTS` = `16`, `SHIFT` = `30`, `SEL_P` = `31`, `SEL_N` = `30`:
  - Shift out two bits per HSTX clock cycle, MSB-first.
  - Each cycle, the shift register advances to the left by two. The most-significant bit is presented to the pin for the first half of that cycle, and the neighbouring bit is presented for the latter half.
- `N_SHIFTS` = `8`, `SHIFT` = `4`, `SEL_P` = `0`, `SEL_N` = `0`:
  - Shift out the least-significant bit in each group of 4 bits, over the course of 8 clock cycles.
  - Each cycle, the shift register advances by to the right by four. The least-significant bit of the shift register is presented to the pin. The bit indices presented to the pin are therefore 0, 4, 8, 12, 16, 20, 24, and 28.
- `N_SHIFTS` = `32`, `SHIFT` = `4`, `SEL_P` = `0`, `SEL_N` = `0`:

- Same as the previous, but repeats the 8-cycle pattern four times before refreshing the shift register.
- Rotating by 32 restores the original value that was popped into the shift register from the FIFO or command expander.

12.11.3.2. Examples: multiple pins

The separation of shift register and bit crossbar allows both zipped and unzipped multi-bit records, once multiple pins are involved. For example, compare these two configurations:

- `N_SHIFTS` = `8`, `SHIFT` = `4`, BIT0.SEL_P = `0`, BIT0.SEL_N = `2`, BIT1.SEL_P = `1`, BIT1.SEL_N = `3`:
  - Each 32-bit word consists of 16 bit-pairs, and a new bit-pair is presented to `BIT0` and `BIT1` twice per cycle.
  - The shift register advances by 4 every cycle, introducing two new bit-pairs to the rightmost four bits of the shift register
- `N_SHIFTS` = `8`, `SHIFT` = `2`, BIT0.SEL_P = `0`, BIT0.SEL_N = `1`, BIT1.SEL_P = `16`, BIT1.SEL_N = `17`:
  - Each 32-bit word consists of a pair of 16-bit values, each of which is shifted to one pin out of `BIT0` and `BIT1` at a rate of two bits per cycle.
  - The shift register advances by two every cycle, introducing a new bit-pair to bits 1:0 for the `BIT0` pin, and also introducing a new bit-pair to bits 17:16 for the `BIT1` pin.

Depending on software needs, it might be preferable to pack together all of the bits output on the same cycle (zipped records), or all of the bits that go through the same pin (unzipped records), so HSTX supports both.

As a final, concrete example, take TMDS (used in DVI): here each 32-bit word contains 3 × 10-bit TMDS symbols, each of which is serialised to a differential pair over the course of 10 TMDS bit times. For performance, it's preferable to make each HSTX clock period equal to *two* TMDS bit periods, by leveraging the DDR capability. A possible configuration would therefore be:

- CSR: `N_SHIFTS` = `5`, `SHIFT` = `2`
- BIT0: `SEL_P` = `0`, `SEL_N` = `1`, `INV` = `0`
- BIT1: `SEL_P` = `0`, `SEL_N` = `1`, `INV` = `1`
- BIT2: `SEL_P` = `10`, `SEL_N` = `11`, `INV` = `0`
- BIT3: `SEL_P` = `10`, `SEL_N` = `11`, `INV` = `1`
- BIT4: `SEL_P` = `20`, `SEL_N` = `21`, `INV` = `0`
- BIT5: `SEL_P` = `20`, `SEL_N` = `21`, `INV` = `1`

The missing piece for TMDS is the clock, which has a period of 10 TMDS bit periods, or 5 HSTX clock periods when shifting two bits per cycle per pin. HSTX has a special-purpose clock generator so that pseudo-clock bits do not have to be packed into the FIFO data stream. The clock generator is covered in the next section.

12.11.4. Clock generator

The clock generator is a counter that provides a periodic signal over the course of `n` HSTX clock cycles, configured by CSR.CLKDIV. The clock period is always an integer number of HSTX clock cycles, in the range 1 to 16 inclusive. The clock generator supports both odd and even periods, using the DDR outputs to support mid-HSTX-cycle output transitions. There is only a single clock generator — to emulate multiple clocks, pack pseudo-clock bits into FIFO data.

The clock generator increments on cycles where the output shift register is shifted. Generally, the clock period will be a divisor of CSR.N_SHIFTS so that clock and data maintain a consistent alignment. In the TMDS example in the previous section, a `CLKDIV` of 5 would be suitable, so that the clock repeats every time the shift register refreshes. This matches the requirement for a TMDS clock period of 10 bit periods, since two bits are transferred every cycle.

The clock generator output is connected to any pin whose `BITx.CLK` bit is set (e.g. BIT0.CLK). To produce differential

clock outputs, connect the clock to two pins, and invert one of them.

The CSR.CLKPHASE field defines the initial phase (count) of the clock generator, configured in units of one half HSTX clock cycle. The clock generator resets whenever CSR.EN is low and holds at this initial phase. Once CSR.EN is set and the output shift register begins to shift, the clock generator advances.

Clock generator output whilst CSR.EN is low is determined by the relation of clock period and initial clock phase: if the initial clock phase is less than one half clock period, then the output is initially low. Otherwise, it is initially high. The clock generator can be thought of as being low for the first half of each generation period, and high for the second half.

The maximum CSR.CLKPHASE is only 15 *half* HSTX clock cycles. The maximum CSR.CLKDIV is 16 *full* HSTX clock cycles: initial phases of greater than or equal to 180 degrees with the maximum clock period require the inversion of the clock using the bit crossbar inversion controls.

Only change CSR.CLKPHASE and CSR.CLKDIV when CSR.EN is low. It is safe to modify them in the same register write that sets `EN` from low to high.

12.11.4.1. Example: centre-aligned clock

When transmitting source-synchronous data, the data sink (the receiver) must not see data transitions too late before or too soon after the active edges of the clock. Violating these setup and hold constraints can lead to undefined operation of the external data sink.

Since the HSTX output delays are all mutually balanced, you can meet these constraints by placing clock transitions halfway between data transitions, known as centre-aligned clocking.

Since this positions the clock with a temporal resolution of one half of a bit time, the maximum data rate is one bit per HSTX clock cycle per pin. Because the clock already uses DDR, you cannot use DDR to increase the data rate. Therefore for all BIT0 through BIT7, `BITx.SEL_N` is equal to `BITx.SEL_P`.

For single-data-rate data, with an active rising edge, use the following clock generator settings:

- CSR.CLKDIV = 1 (1 HSTX clock period)
- CSR.CLKPHASE = 1 (1/2 HSTX clock period)

The clock is delayed by half an HSTX cycle, to offset it from the launch of the first data.

For single-data-rate data, with an active *falling* edge, use the following clock generator settings:

- CSR.CLKDIV = 1 (1 HSTX clock period)
- CSR.CLKPHASE = 2 (1 HSTX clock period)

Alternatively, you could use the same settings as an active-rising edge clock, with the clock output inverted via the bit crossbar configuration.

For double-data-rate data, with active rising and active falling edges, use the following clock generator settings:

- CSR.CLKDIV = 2 (2 HSTX clock period)
- CSR.CLKPHASE = 1 (1/2 HSTX clock period)

In all three cases, the data rate is the same, at 1 bit per HSTX clock cycle, per pin.

12.11.5. Command expander

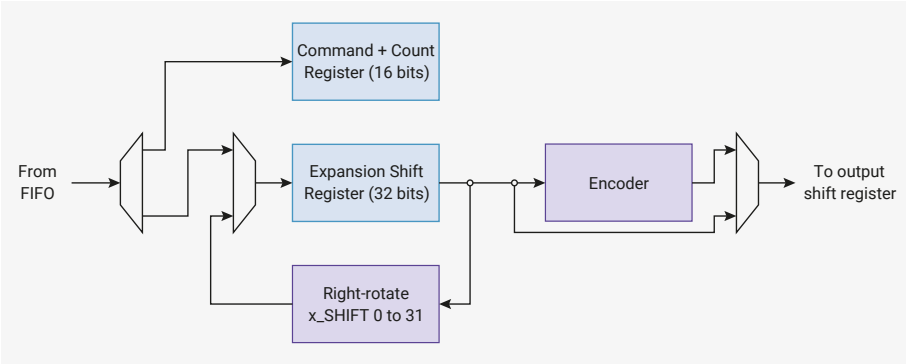

Figure 128. A mixture of commands and data are popped from the FIFO. Data can be repeated or shifted through the expansion shift register, and optionally passed through an encoder before passing on to the output shift register.

The command expander can be inserted inline between the data FIFO and the output shift register to manipulate the stream of data words. In general, the output stream is larger than the input stream, hence the name expander. The commander expander is enabled by setting CSR.EXPAND_EN. Only modify this field when CSR.EN is low. It is safe to modify this field in the same register write that sets `EN` from low to high. When the command expander is disabled, data passes directly from the data FIFO to the output shift register without being modified by the expander.

When the command expander is enabled, the data FIFO carries a mixture of data and commands for the expander. Each command consists of a 4-bit opcode and a 12-bit length, packed in the 16 LSBs of a data FIFO word, with the opcode in bits 15 through 12, and the length in bits 11 through 0. The available commands are:

- `0x0`: `RAW`
- `0x1`: `RAW_REPEAT`
- `0x2`: `TMDS`
- `0x3`: `TMDS_REPEAT`
- `0xf`: `NOP`

When the HSTX is first enabled, if the command expander is enabled, it expects the first word in the data FIFO to be a command. If this command is not a `NOP`, it will be followed by some amount of data, then another command. Operation continues in this manner, with runs of data interspersed with commands. A command always acts as a prefix to the data that follows it in the FIFO.

The count field determines the number of words output by this command to the output shift register downstream, from 1 to 4095. A count of 0 is reserved to mean "infinite". The number of words that this command reads from the data FIFO in order to produce the specified quantity of downstream data depends on the command and the EXPAND_SHIFT.ENC_N_SHIFTS and EXPAND_SHIFT.RAW_N_SHIFTS register fields.

The expansion shift register always pops from the FIFO once at the beginning of the command. After this point, commands bearing the `x_REPEAT` suffix continue to circulate the same contents through the shift register, rotating right by EXPAND_SHIFT.ENC_SHIFT or EXPAND_SHIFT.RAW_SHIFT each time the output shift register pulls new data from the command expander. Use a shift of 0 to repeat identical data without shifting. This is useful, for example, for transmitting runs of the same TMDS control symbol during horizontal blanking periods in DVI.

RP2350 only implements a TMDS encoder, reserving the remaining opcode space for additional encoders in the future. `RAW` and `RAW_REPEAT` commands bypass the encoder. `TMDS` and `TMDS_REPEAT` commands are TMDS-encoded before being passed to the output shift register. `NOP` commands have no data, therefore whether they bypass the encoder or not is a philosophical question beyond the scope of this datasheet.

The EXPAND_SHIFT register has two copies for each of its fields. Fields prefixed with `RAW_` are used for `RAW` and `RAW_REPEAT` commands. All other commands use fields prefixed with `ENC_`, which pass through the encoder. For example, in DVI, TMDS control symbols using `RAW_REPEAT` commands may be unshifted. Pixel data using TMDS commands may be shifted out one pixel at a time, so it is useful to have banked shift controls.

The EXPAND_SHIFT.ENC_N_SHIFTS and EXPAND_SHIFT.RAW_N_SHIFTS fields control how often the expansion shift register is refilled for encoded and raw commands respectively. `x_REPEAT` commands ignore these fields since they never refill from the FIFO, and function similarly to the CSR.N_SHIFTS field that controls the output shift register.

The command expander can only pop from the data FIFO once per cycle, so heavy use of commands (particularly `NOP`

commands) can impact HSTX throughput. For use cases that output from the HSTX on every cycle, configure the output shift register with CSR.N_SHIFTS > 1. This is required because the command expander cannot output data on the cycle where it pops a command from the FIFO, so the expansion shift register is empty for at least one cycle.

12.11.6. PIO-to-HSTX coupled mode

HSTX can connect up to 8 PIO pin outputs to the bit crossbar. Only use the bit crossbar when `clk_hstx` connects directly to `clk_sys` (CLK_HSTX_CTRL.AUXSRC must select `clk_sys`).

NOTE

> Running the two clocks at the same frequency is not sufficient. You must select `clk_sys` directly.

To enable coupled mode, set CSR.COUPLED_MODE. The `COUPLED_SEL` field in the same register selects the PIO instance, 0 through 2, to couple to HSTX. When coupled mode is enabled, IO outputs 12 through 19 inclusive on the selected PIO instance appear at bit crossbar `PSEL_N` and `PSEL_P` indices 31:24, replacing the most significant 8 bits of the output shift register from the point of view of the bit crossbar.

This mode allows PIO programs to make use of the HSTX's DDR outputs. You can use this mode to drive a clock at the full system clock rate or to position clock transitions relative to data transitions with half-system-clock-cycle resolution.

The PIO outputs used for couple mode are always bits 19 through 12 of the pin outputs driven from that GPIO, independent of GPIOBASE. When `GPIOBASE` is 0, the PIO outputs used for coupled mode are those that would normally appear on the `HSTX` pins. When `GPIOBASE` is 16, this uses the PIO outputs that would appear on GPIOs 28 through 35.

The operation of PIO is not affected in any way by coupled mode being enabled.

Outputs presented through the HSTX coupled mode interface have one additional system clock cycle of delay compared to those presented directly from PIO to the pads.

12.11.7. List of control registers

The control registers start at a base address of `0x400c0000` (defined as HSTX_CTRL_BASE in the SDK). They are accessed through an asynchronous bus crossing, so each bus access takes several cycles, the exact figure depending on the ratio of `clk_sys` and `clk_hstx`.

Table 1253. List of HSTX_CTRL registers

Offset	Name	Info
0x00	CSR	
0x04	BIT0	Data control register for output bit 0
0x08	BIT1	Data control register for output bit 1
0x0c	BIT2	Data control register for output bit 2
0x10	BIT3	Data control register for output bit 3
0x14	BIT4	Data control register for output bit 4
0x18	BIT5	Data control register for output bit 5
0x1c	BIT6	Data control register for output bit 6
0x20	BIT7	Data control register for output bit 7
0x24	EXPAND_SHIFT	Configure the optional shifter inside the command expander
0x28	EXPAND_TMDS	Configure the optional TMDS encoder inside the command expander

HSTX_CTRL: CSR Register

Offset: 0x00

Table 1254. CSR Register

Bits	Description	Type	Reset
31:28	**CLKDIV**: Clock period of the generated clock, measured in HSTX clock cycles. Can be odd or even. The generated clock advances only on cycles where the shift register shifts. For example, a clkdiv of 5 would generate a complete output clock period for every 5 HSTX clocks (or every 10 half-clocks). A CLKDIV value of 0 is mapped to a period of 16 HSTX clock cycles.	RW	0x1
27:24	**CLKPHASE**: Set the initial phase of the generated clock. A CLKPHASE of 0 means the clock is initially low, and the first rising edge occurs after one half period of the generated clock (i.e. CLKDIV/2 cycles of clk_hstx). Incrementing CLKPHASE by 1 will advance the initial clock phase by one half clk_hstx period. For example, if CLKDIV=2 and CLKPHASE=1: * The clock will be initially low * The first rising edge will be 0.5 clk_hstx cycles after asserting first data * The first falling edge will be 1.5 clk_hstx cycles after asserting first data This configuration would be suitable for serialising at a bit rate of clk_hstx with a centre-aligned DDR clock. When the HSTX is halted by clearing CSR_EN, the clock generator will return to its initial phase as configured by the CLKPHASE field. Note CLKPHASE must be strictly less than double the value of CLKDIV (one full period), else its operation is undefined.	RW	0x0
23:21	Reserved.	-	-
20:16	**N_SHIFTS**: Number of times to shift the shift register before refilling it from the FIFO. (A count of how many times it has been shifted, **not** the total shift distance.) A register value of 0 means shift 32 times.	RW	0x05
15:13	Reserved.	-	-
12:8	**SHIFT**: How many bits to right-rotate the shift register by each cycle. The use of a rotate rather than a shift allows left shifts to be emulated, by subtracting the left-shift amount from 32. It also allows data to be repeated, when the product of SHIFT and N_SHIFTS is greater than 32.	RW	0x06
7	Reserved.	-	-
6:5	**COUPLED_SEL**: Select which PIO to use for coupled mode operation.	RW	0x0

Bits	Description	Type	Reset
4	**COUPLED_MODE**: Enable the PIO-to-HSTX 1:1 connection. The HSTX must be clocked **directly** from the system clock (not just from some other clock source of the same frequency) for this synchronous interface to function correctly. When COUPLED_MODE is set, BITx_SEL_P and SEL_N indices 24 through 31 will select bits from the 8-bit PIO-to-HSTX path, rather than shifter bits. Indices of 0 through 23 will still index the shift register as normal. The PIO outputs connected to the PIO-to-HSTX bus are those same outputs that would appear on the HSTX-capable pins if those pins' FUNCSELs were set to PIO instead of HSTX. For example, if HSTX is on GPIOs 12 through 19, then PIO outputs 12 through 19 are connected to the HSTX when coupled mode is engaged.	RW	0x0
3:2	Reserved.	-	-
1	**EXPAND_EN**: Enable the command expander. When 0, raw FIFO data is passed directly to the output shift register. When 1, the command expander can perform simple operations such as run length decoding on data between the FIFO and the shift register. Do not change CXPD_EN whilst EN is set. It's safe to set CXPD_EN simultaneously with setting EN.	RW	0x0
0	**EN**: When EN is 1, the HSTX will shift out data as it appears in the FIFO. As long as there is data, the HSTX shift register will shift once per clock cycle, and the frequency of popping from the FIFO is determined by the ratio of SHIFT and SHIFT_THRESH. When EN is 0, the FIFO is not popped. The shift counter and clock generator are also reset to their initial state for as long as EN is low. Note the initial phase of the clock generator can be configured by the CLKPHASE field. Once the HSTX is enabled again, and data is pushed to the FIFO, the generated clock's first rising edge will be one half-period after the first data is launched.	RW	0x0

HSTX_CTRL: BIT0, BIT1, …, BIT6, BIT7 Registers

Offsets: 0x04, 0x08, …, 0x1c, 0x20

Description

Data control register for output bit *n*

Table 1255. BIT0, BIT1, …, BIT6, BIT7 Registers

Bits	Description	Type	Reset
31:18	Reserved.	-	-
17	**CLK**: Connect this output to the generated clock, rather than the data shift register. SEL_P and SEL_N are ignored if this bit is set, but INV can still be set to generate an antiphase clock.	RW	0x0
16	**INV**: Invert this data output (logical NOT)	RW	0x0
15:13	Reserved.	-	-
12:8	**SEL_N**: Shift register data bit select for the second half of the HSTX clock cycle	RW	0x00

Bits	Description	Type	Reset
7:5	Reserved.	-	-
4:0	**SEL_P**: Shift register data bit select for the first half of the HSTX clock cycle	RW	0x00

HSTX_CTRL: EXPAND_SHIFT Register

Offset: 0x24

Description

Configure the optional shifter inside the command expander

Table 1256. EXPAND_SHIFT Register

Bits	Description	Type	Reset
31:29	Reserved.	-	-
28:24	**ENC_N_SHIFTS**: Number of times to consume from the shift register before refilling it from the FIFO, when the current command is an encoded data command (e.g. TMDS). A register value of 0 means shift 32 times.	RW	0x01
23:21	Reserved.	-	-
20:16	**ENC_SHIFT**: How many bits to right-rotate the shift register by each time data is pushed to the output shifter, when the current command is an encoded data command (e.g. TMDS).	RW	0x00
15:13	Reserved.	-	-
12:8	**RAW_N_SHIFTS**: Number of times to consume from the shift register before refilling it from the FIFO, when the current command is a raw data command. A register value of 0 means shift 32 times.	RW	0x01
7:5	Reserved.	-	-
4:0	**RAW_SHIFT**: How many bits to right-rotate the shift register by each time data is pushed to the output shifter, when the current command is a raw data command.	RW	0x00

HSTX_CTRL: EXPAND_TMDS Register

Offset: 0x28

Description

Configure the optional TMDS encoder inside the command expander

Table 1257. EXPAND_TMDS Register

Bits	Description	Type	Reset
31:24	Reserved.	-	-
23:21	**L2_NBITS**: Number of valid data bits for the lane 2 TMDS encoder, starting from bit 7 of the rotated data. Field values of 0 → 7 encode counts of 1 → 8 bits.	RW	0x0
20:16	**L2_ROT**: Right-rotate applied to the current shifter data before the lane 2 TMDS encoder.	RW	0x00
15:13	**L1_NBITS**: Number of valid data bits for the lane 1 TMDS encoder, starting from bit 7 of the rotated data. Field values of 0 → 7 encode counts of 1 → 8 bits.	RW	0x0
12:8	**L1_ROT**: Right-rotate applied to the current shifter data before the lane 1 TMDS encoder.	RW	0x00